ASSESSMENT TOOL 1

Course Name: Database Management Systems **Course Code:** CSA0504

CO1: *Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.*

Q1. Explain schema and instance with examples. Distinguish between logical schema, physical schema, and view schema.

Schema

A schema is the overall logical design (structure) of a database. It defines the tables, fields, data types, relationships, and constraints, and it does not change frequently — only when the database is redesigned. For example, in a College database, the schema may define: STUDENT(Roll_No, Name, Dept, Year) and COURSE(Course_Id, Course_Name, Credits).

Instance

An instance (also called a database state or snapshot) is the actual data stored in the database at a particular moment in time. Since data is inserted, updated, or deleted regularly, the instance keeps changing while the schema stays fixed. For example, at 10 AM the STUDENT table may contain rows such as (101, 'Arun', 'CSE', 2) and (102, 'Divya', 'IT', 3) — this specific set of rows is one instance of the schema.

Difference between Logical, Physical, and View Schema

Basis	Logical Schema	Physical Schema	View Schema
Level	Conceptual level of the three-schema architecture	Internal / physical level of the three-schema architecture	External level of the three-schema architecture
Describes	What data is stored and the relationships among data (tables, attributes, keys, constraints)	How data is physically stored on storage devices (file organization, indexing, access paths)	How individual users or user groups see a customized portion of the database
Concern	Data structure and relationships, independent of physical storage	Storage efficiency, indexing method, and data access speed	User-specific presentation and data security/hiding

Who defines it	Database designer / DBA	DBA / storage engineer	Application designer for a specific class of users
Example	Defining EMPLOYEE(Emp_id, Name, Salary, Dept_id) as a table with a primary key	Storing the EMPLOYEE table as an indexed sequential file on disk with a B-tree index on Emp_id	A view showing only Name and Dept for the HR clerk, hiding Salary

In short: **the logical schema defines what data exists**, **the physical schema defines how it is stored**, and **the view schema defines what part of it a given user is allowed to see**.

Q2. Identify the entities, attributes, and relationships for a Bank Management System and construct its ER diagram.

Entities and their Attributes

- **CUSTOMER** — Customer_Id (PK), Name, Address, Phone, Email, DOB
- **ACCOUNT** — Account_No (PK), Account_Type, Balance, Opened_Date
- **BRANCH** — Branch_Id (PK), Branch_Name, Location, IFSC_Code
- **EMPLOYEE** — Employee_Id (PK), Name, Designation, Salary
- **LOAN** — Loan_No (PK), Loan_Type, Amount, Interest_Rate
- **TRANSACTION** — Transaction_Id (PK), Date, Amount, Type (Deposit/Withdrawal)

Relationships

- **A CUSTOMER** holds one or more ACCOUNTs (1:M) — a customer can have several accounts, but each account belongs to one customer.
- **An ACCOUNT** maintained-at a BRANCH (M:1) — many accounts are managed at one branch.
- **An EMPLOYEE** works-at a BRANCH (M:1) — many employees work at one branch.
- **A CUSTOMER** applies-for a LOAN (1:M) — a customer may take multiple loans.
- **An ACCOUNT** generates a TRANSACTION (1:M) — one account can have many transactions.

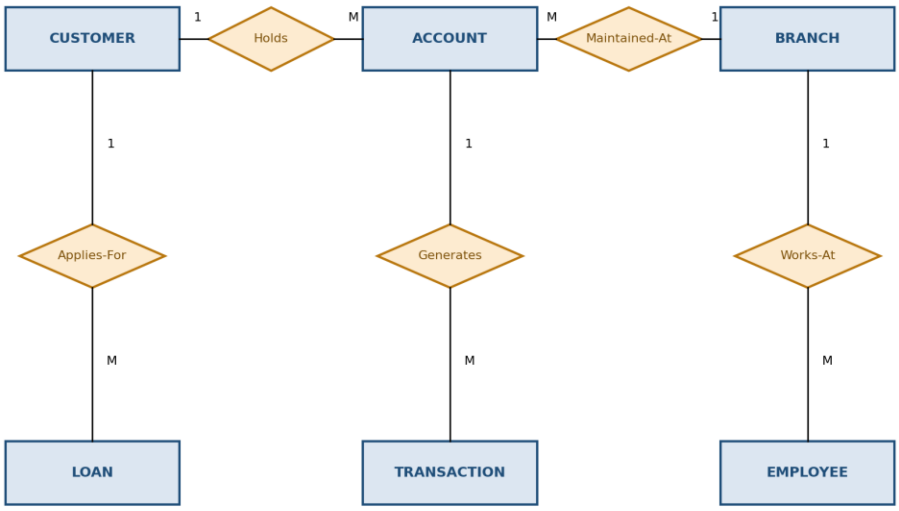

Fig. Q2 — ER Diagram of the Bank Management System
(rectangles = entities, diamonds = relationships; 1 and M denote cardinality)

Q3. Explain the Extended ER (EER) model. Describe the concepts of generalization, specialization, and aggregation.

Extended ER (EER) Model

The Extended Entity-Relationship (EER) model builds on the basic ER model by adding semantic concepts such as generalization, specialization, inheritance, and aggregation. It is used to model complex applications more accurately by allowing entity types to be organized into superclass/subclass hierarchies and by representing higher-level relationships. EER diagrams are especially useful when different types of an entity share some attributes but also have their own distinct attributes.

Generalization

Generalization is a bottom-up process in which two or more lower-level entities that share common attributes are combined to form a single higher-level (generalized) entity. It emphasizes the similarities among entities and hides their differences. For example, entities CAR and TRUCK, which share attributes like Vehicle_No and Model, can be generalized into a single entity VEHICLE.

Specialization

Specialization is the reverse, top-down process in which a higher-level entity is divided into two or more lower-level entities based on distinguishing characteristics. Each sub-entity inherits the attributes of the parent entity and also has its own specific attributes. For example, the entity EMPLOYEE can be

specialized into TEACHING_STAFF and NON_TEACHING_STAFF, where TEACHING_STAFF additionally has a Subject_Specialization attribute.

Aggregation

Aggregation is a concept used to model a relationship between a relationship set and an entity set, treating a whole relationship as a higher-level abstract entity so it can participate in further relationships. It is used when a relationship itself needs to be related to another entity. For example, the relationship "WORKS_ON" between EMPLOYEE and PROJECT can be aggregated into a single unit, which then participates in a relationship "MONITORS" with the entity MANAGER, since a manager monitors the employee's work on a project as a whole, not the employee or project individually.

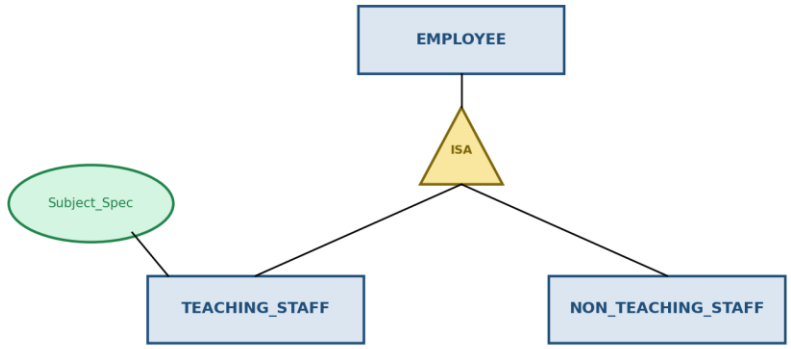

Fig. Q3 — Specialization of EMPLOYEE into TEACHING_STAFF and NON_TEACHING_STAFF using the ISA (is-a) triangle notation.

Q4. Design an EER diagram for a Hospital Management System incorporating inheritance and weak entities.

Entities, Inheritance, and Weak Entity Identified

- **PERSON (superclass)** — Person_Id (PK), Name, Gender, Phone, Address — specialized into DOCTOR and PATIENT using generalization/specialization (inheritance).
- **DOCTOR (subclass of PERSON)** — Doctor_Id, Specialization, Consultation_Fee.
- **PATIENT (subclass of PERSON)** — Patient_Id, Blood_Group, Admission_Date.
- **DEPARTMENT** — Dept_Id (PK), Dept_Name, Location.

- **APPOINTMENT (regular entity)** — Appointment_Id (PK), Date, Time — connects DOCTOR and PATIENT.
- **DEPENDENT (weak entity)** — Dependent_Name (partial key), Relationship, Age — cannot exist without a PATIENT; identified only through Patient_Id + Dependent_Name.

Here, inheritance is shown by PERSON generalizing common attributes of DOCTOR and PATIENT, who inherit Name, Gender, Phone, and Address. The weak entity DEPENDENT has no independent primary key of its own; it is existence-dependent on the identifying (owner) entity PATIENT through an identifying relationship (shown with a double-diamond) and is drawn with a double-outlined rectangle.

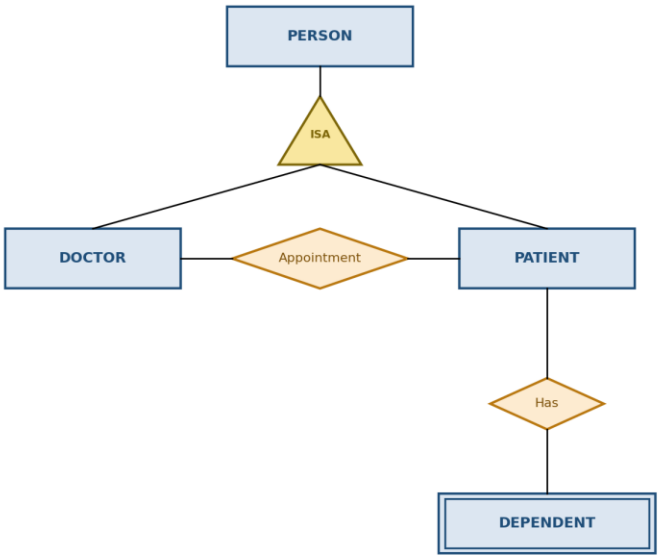

Fig. Q4 — EER Diagram of the Hospital Management System. PERSON generalizes DOCTOR and PATIENT (inheritance); DEPENDENT is a weak entity (double rectangle) linked to PATIENT via an identifying relationship (double diamond).

Q5. Explain the role of a DBA (Database Administrator) and the responsibilities associated with the position.

A Database Administrator (DBA) is the person (or team) responsible for the overall management, control, and maintenance of a database system, ensuring it remains secure, consistent, efficient, and available to authorized users at all times.

Key Responsibilities of a DBA

- **Schema Definition:** Creating the original database schema by writing a set of DDL (Data Definition Language) statements that define tables, attributes, and constraints.
- **Storage Structure and Access Method Definition:** Deciding how data is stored physically and choosing appropriate indexing and file organization methods for efficient access.
- **Schema and Physical Organization Modification:** Carrying out changes to the schema or physical structure as organizational or performance needs evolve.
- **Granting Authorization for Data Access:** Controlling which users or roles can read, insert, update, or delete specific data through GRANT and REVOKE privileges, ensuring data security.
- **Routine Maintenance:** Performing periodic backups, monitoring disk space, applying software patches, and ensuring recovery procedures are in place for crashes or disk failures.
- **Ensuring Data Integrity and Consistency:** Enforcing constraints (primary key, foreign key, check constraints) so that the stored data remains accurate and reliable.
- **Performance Monitoring and Tuning:** Analyzing query performance, optimizing indexes, and tuning the database engine to maintain fast response times as data volume grows.
- **Coordinating with Users:** Liaising between end users, application developers, and management to gather requirements and resolve database-related issues.

In summary, the DBA acts as the custodian of the database — balancing the needs of security, integrity, performance, and availability so that the database serves the organization reliably.